

Unidad III — Clase 1

Patrones de diseño: el catálogo GoF

Creacionales, estructurales y de comportamiento, sobre Java 21 / Spring Boot 4 y React 19 + TypeScript.

Caso transversal: FitZone Sports.

4 h

Clase teórico-aplicada

23

Patrones del catálogo

13

En profundidad, con código

2026

Plan de estudio 2024

Agenda

Cinco bloques.

0 **Qué es un patrón y qué no lo es.** Origen, anatomía de una ficha, los tres principios que sostienen el catálogo.

1 **Creacionales.** Factory Method, Abstract Factory, Builder, Singleton, Prototype.

2 **Estructurales.** Adapter, Decorator, Facade, Proxy, Composite (+ Bridge y Flyweight).

3 **Comportamiento.** Strategy, Observer, Command, Template Method, State, Chain of Responsibility.

4 **Combinar, criticar y decidir.** Patrones en el framework, cuándo NO usarlos, vigencia del catálogo.

0

BLOQUE 0 · 25 MINUTOS

Qué es un patrón de diseño

Y, sobre todo, qué no lo es: ni una receta, ni una biblioteca, ni una medalla que se cuelga al código.

El problema no es escribir el código; es volver a tocarlo

Un diseño sin patrones

- El requerimiento nuevo obliga a editar código que ya funcionaba.
- Cada condicional nuevo se agrega a un if/else que ya tiene siete ramas.
- No se puede probar una regla sin levantar la base de datos.
- Nadie sabe nombrar lo que hace esa clase sin leerla entera.

Un diseño con patrones

- El requerimiento nuevo se resuelve agregando una clase, no editando diez.
- La variación está encapsulada detrás de una interfaz.
- Cada regla se prueba sola, con dobles de prueba.
- El nombre del patrón comunica la intención en una palabra.

PERO La columna derecha no es gratis: cada patrón agrega indirección, clases y nombres nuevos. Un patrón mal elegido produce exactamente el mismo daño que el código que venía a arreglar, con la diferencia de que ahora nadie se anima a tocarlo porque «tiene un patrón».

REGLA DE LA CÁTEDRA Un patrón se justifica cuando hay una variación real y conocida, no una imaginada. Si el segundo caso todavía no existe, el patrón es deuda técnica anticipada.

De la arquitectura de edificios al catálogo de 1994



DEFINICIÓN «Cada patrón describe un problema que ocurre una y otra vez, y el núcleo de su solución, de modo que esa solución se pueda usar un millón de veces sin hacerla nunca dos veces de la misma forma» — Christopher Alexander, 1977.

1

Un patrón es un nombre

«Usemos un Strategy acá» transporta media hora de diseño en tres palabras. El vocabulario compartido es la mitad del valor.

2

Un patrón es una estructura

Roles y relaciones entre clases u objetos. No es una clase concreta ni un fragmento de código para copiar.

3

Un patrón son sus consecuencias

Qué gana y qué pierde el diseño. La ficha de GoF dedica más espacio a esto que a la solución.

OJO GoF escribieron para C++ y Smalltalk de 1994, sin lambdas, sin genéricos, sin inyección de dependencias y sin módulos. Varias de sus soluciones hoy son una línea del lenguaje. Volveremos sobre esto en el Bloque 4.

Cómo se lee una ficha de patrón

GoF define 13 secciones por patrón. Estas cinco son las que hay que poder responder en un examen oral.

1 **Intención.** Una oración. Qué problema resuelve y la solución.

2 **Motivación.** Explica en más detalle el problema y la solución que brinda el patrón.

3 **Estructura y participantes.** Los roles (Contexto, Estrategia, Producto Concreto...) y cómo colaboran. El diagrama, no el código.

4 **Consecuencias.** Qué gana el diseño y qué paga. Siempre paga algo: clases, indirección, dificultad para depurar.

5 **Patrones relacionados.** Con qué se combina y con qué se confunde. Adapter, Decorator, Proxy y Facade tienen casi la misma estructura y distinta intención.

Los tres principios que sostienen todo el catálogo

Si se olvidan los 23 patrones y se quedan con estos tres enunciados, van a reinventar la mitad del catálogo solos.

1

Programar contra una interfaz, no contra una implementación

El cliente depende del tipo abstracto; la clase concreta se decide en otro lado (una fábrica, el contenedor IoC, un archivo de configuración).

Consecuencia: se puede sustituir la implementación sin tocar al cliente — y por eso se puede usar un doble en los tests.

2

Favorecer la composición de objetos sobre la herencia de clases

La herencia fija el comportamiento al compilar y expone al hijo las tripas del padre. La composición lo decide en ejecución y solo a través de una interfaz pública.

Consecuencia: comportamiento intercambiable en caliente y sin explosión combinatoria de subclases.

3

Encapsular lo que varía

Identificar qué parte del sistema cambia con cada requerimiento nuevo y separarla de la que permanece estable.

Consecuencia: los cambios futuros quedan confinados. Es, literalmente, el principio abierto/cerrado dicho en criollo.

EL CUARTO PRINCIPIO, NO ESCRITO Ninguno de los tres dice «siempre». Aplicados sin una variación real producen una arquitectura de interfaces con una sola implementación cada una: el olor se llama «abstracción especulativa».

Los 23 patrones, clasificados

Propósito (qué hace el patrón) × alcance (si la variación se resuelve con herencia de clases o con composición de objetos).

Alcance	Creacionales (5)	Estructurales (7)	Comportamiento (11)
Clase (herencia)	Factory Method ★	Adapter de clase	Interpreter · Template Method ★
Objeto (composición)	Abstract Factory ★ · Builder ★ Prototype · Singleton ★	Adapter ★ · Bridge · Composite ★ Decorator ★ · Facade ★ · Flyweight · Proxy ★	Chain of Responsibility ★ · Command ★ · Iterator Mediator · Memento · Observer ★ · State ★ Strategy ★ · Visitor

★ Los trece marcados se ven hoy con código, diagrama y contraejemplo. Los diez restantes quedan en tabla de referencia al cierre de cada bloque y en el cheat-sheet.

Creacionales

Se ocupan de CÓMO y CUÁNDO se crea un objeto. Sacan el «new» de donde estorba y esconden qué clase concreta se instancia.

Son los que más envejecieron: el contenedor IoC se quedó con buena parte de su trabajo.

Estructurales

Se ocupan de CÓMO SE COMPONEN los objetos en estructuras más grandes. Casi todos envuelven a otro objeto; cambia para qué lo envuelven.

Son los que más aparecen en las fronteras del sistema.

Comportamiento

Se ocupan de CÓMO SE REPARTE la responsabilidad y cómo se comunican los objetos. Es el grupo más grande y el más usado.

Son los que resuelven requerimientos de negocio, no de infraestructura.

Patronitis: el catálogo usado como decoración

1

Abstracción especulativa

Una interfaz por si algún día hay otra implementación. El día no llega nunca y el archivo extra sí.

2

Fábrica de fábricas

Tres niveles de indirección para instanciar una clase que nunca tuvo variantes.

3

Patrón como nombre

Una clase que se llama «PedidoStrategy» pero tiene un único método con un if adentro. El nombre no es el patrón.

4

Reimplementar el framework

Un Singleton a mano dentro de una aplicación Spring, que ya administra el ciclo de vida de sus beans.

5

Patrón mal identificado

Un State documentado como Strategy. La estructura coincide; la intención (y el examen) no.

6

Patrón inmovilizado

Nadie toca la clase porque «tiene un patrón». La indirección, en vez de proteger el cambio, lo bloquea.

LA REGLA DE TRES Un caso: escribilo directo. Dos casos: duplicá y aguantá la incomodidad; todavía no sabés cuál es el eje de variación. Tres casos: ahí sí, abstraé — recién con el tercero se ve qué varía de verdad. Introducir el patrón antes del tercer caso es adivinar.

Los patrones que ya usaron sin saberlo

Todo el stack de la Unidad II está construido con el catálogo. Reconocerlos ahí es la mitad del trabajo de esta unidad.

Patrón	En Java / JDK	En Spring	En React / TypeScript
Factory Method	List.of(), Optional.of()	BeanFactory.getBean()	createContext(), createRoot()
Builder	StringBuilder, Stream.Builder	UriComponentsBuilder	encadenado de Zod / query builders
Singleton	Runtime.getRuntime()	todo bean de scope singleton	módulo ES con estado (store de Zustand)
Adapter	Arrays.asList()	HandlerAdapter	adaptadores de cliente HTTP
Decorator	BufferedReader sobre Reader	BeanPostProcessor, TransactionInterceptor	HOC, wrappers de hooks, middlewares
Proxy	java.lang.reflect.Proxy	@Transactional, @Cacheable, lazy de Hibernate	Proxy de JS, stores reactivos de Vue
Observer	Flow (java.util.concurrent)	ApplicationEventPublisher	useEffect + suscripción, signals
Strategy	Comparator	PasswordEncoder, cualquier List<T> inyectada	función pasada por prop
Template Method	AbstractList	JdbcTemplate, RestTemplate	ciclo de vida de un componente de clase
Command	Runnable, Callable	tareas de @Async	acciones de Redux, mutaciones de TanStack Query

PREGUNTA ABIERTA ¿Por qué anotar un método con `@Transactional` y llamarlo desde otro método de la MISMA clase no abre una transacción? La respuesta está en esta tabla y la vemos en el Bloque 2.

1

BLOQUE 1 · 55 MINUTOS

Patrones creacionales

Cinco maneras de responder a la misma pregunta incómoda: ¿quién decide qué clase concreta se instancia, y dónde vive esa decisión?

¿Qué tiene de malo un «new»?

ACOPLADO

```
public Comprobante cobrar(Reserva r) {  
    // la clase concreta queda fija al compilar  
    var pasarela = new PasarelaMercadoPago(  
        "APP_USR-8f3..."); // y la credencial  
    var pago = pasarela.debitar(r.total());  
    return new Comprobante(pago);  
}  
  
// Este método no se puede probar sin red,  
// sin credencial y sin MercadoPago arriba.
```

DESACOPLADO

```
private final PasarelaPago pasarela; // interfaz  
  
public Comprobante cobrar(Reserva r) {  
    var pago = pasarela.debitar(r.total());  
    return new Comprobante(pago);  
}  
  
// El test construye el servicio con un doble.  
// Pero alguien, en algún lado, sigue teniendo  
// que decidir cuál pasarela se instancia.
```

LA PREGUNTA DEL BLOQUE «Alguien, en algún lado». Los cinco patrones creacionales son cinco respuestas distintas a dónde vive ese alguien y cuánta información necesita para decidir.

Lo decide el contenedor IoC

Caso normal en Spring o NestJS: la decisión es de configuración y se toma al arrancar. No hace falta ningún patrón creacional.

Lo decide un dato de ejecución

«Cobrar con la pasarela que eligió el socio». El contenedor no lo sabe al arrancar. Aquí entran Factory Method y Abstract Factory.

El objeto es complejo de armar

Muchos campos, algunos opcionales, invariantes que validar. Aquí entra Builder.

Los cinco creacionales, en una tabla

Todos esconden la clase concreta. Se diferencian por qué parte del proceso de creación encapsulan.

Patrón	Qué encapsula	Señal de que lo necesitás	Caso en FitZone
Factory Method	Qué clase concreta se instancia, decidido por una subclase o por un parámetro.	Un switch o if/else que hace new de distintas clases según un código o un enum.	Crear el notificador según el canal elegido por el socio (push, e-mail, SMS).
Abstract Factory	Una familia completa de productos que deben ser coherentes entre sí.	Dos o más fábricas que siempre se eligen juntas; mezclarlas produce un estado inválido.	Proveedor de pagos: pasarela + validador de tarjeta + generador de comprobante, por país.
Builder	El proceso de construcción paso a paso de un objeto complejo.	Un constructor con seis o más parámetros, o varios constructores que se diferencian en nulls.	Armar una Reserva: sede, cancha, franja, socio, cupón, política de cancelación, origen.
Prototype	La creación por copia de un objeto ya configurado.	Construir desde cero cuesta más que clonar un ejemplar bien armado.	Generar la grilla semanal de una sede copiando la plantilla de la semana tipo.
Singleton	Que exista una sola instancia y un punto global de acceso a ella.	Casi nunca. En una aplicación con contenedor IoC, el contenedor ya resuelve esto mejor.	Ninguno propio: los beans de Spring ya son singletons administrados.

PRIMER FILTRO Antes de escribir una fábrica, preguntarse: ¿la decisión depende de un dato que solo se conoce en ejecución? Si la respuesta es no, la fábrica sobra: el contenedor IoC ya lo resuelve con perfiles o con configuración.

PARA EL TRABAJO INTEGRADOR En FitZone, el creacional que casi seguro van a necesitar es Builder (la Reserva tiene demasiados campos). Factory Method aparece si implementan más de un canal de notificación. Abstract Factory, probablemente no: y decir por qué no también suma.

Factory Method: delegar en una subclase (o en un mapa) qué se crea

Intención: definir una interfaz para crear un objeto, dejando que otra clase decida cuál de las concretas se instancia.

ANTES · la decisión repartida

```
public void avisar(Socio s, String canal, String m) {
    switch (canal) {
        case "PUSH" -> new FirebasePush().enviar(s, m);
        case "EMAIL" -> new SmtplibMailer().enviar(s, m);
        case "SMS" -> new TwilioSms().enviar(s, m);
        default -> throw new IllegalArgumentException(canal);
    }
}

// Cada canal nuevo edita este método.
// Y el método ya sabe demasiado: conoce
// Firebase, SMTP y Twilio al mismo tiempo.
```

DESPUÉS · registro por mapa

```
public interface Notificador {
    Canal canal(); // se autodescribe
    void enviar(Socio s, String mensaje);
}

@Component
public class FabricaNotificadores {
    private final Map<Canal, Notificador> porCanal;

    FabricaNotificadores(List<Notificador> todos) {
        this.porCanal = todos.stream().collect(
            toMap(Notificador::canal, identity()));
    }

    public Notificador para(Canal c) {
        return Optional.ofNullable(porCanal.get(c))
            .orElseThrow(() -> new CanalNoSoportado(c));
    }
}
```

EL CANAL NUEVO Agregar WhatsApp es agregar una clase anotada con `@Component` que implementa `Notificador`. Ningún archivo existente se edita: eso, y no la cantidad de clases, es el principio abierto/cerrado funcionando.

MATIZ El Factory Method canónico de GoF usa herencia: una subclase de Creador por producto. La versión de arriba es composición y en la práctica reemplazó a la original — pero si en el parcial se pide «el Factory Method de GoF», el diagrama esperado es el de herencia. Enlace con Unidad II: esto es el Caso 3 (RutaYa · AsignadorDeEnvios) con la `List<T>` inyectada.

Abstract Factory: cuando los productos tienen que ser coherentes entre sí

Intención: proveer una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.

1 El síntoma

Hay dos o tres objetos que siempre se eligen juntos, y elegir mal la combinación no da un error de compilación: da un error de negocio en producción.

2 La solución

Una interfaz de fábrica con un método por producto. Cada implementación devuelve la familia completa y consistente. El cliente pide la fábrica, no los productos sueltos.

3 El costo

Agregar un producto nuevo a la familia obliga a tocar la interfaz de fábrica y TODAS sus implementaciones. Abstract Factory es rígido en esa dirección, y lo es a propósito.

HIPÓTESIS · FitZone se expande a Uruguay

```
public interface ProveedorDePagos {  
    PasarelaPago      pasarela();  
    ValidadorFiscal   validador();  
    GeneradorComprobante comprobante();  
}  
  
@Component class ProveedorArgentina implements ProveedorDePagos { /* MercadoPago + AFIP + FacturaAR */ }  
@Component class ProveedorUruguay implements ProveedorDePagos { /* Plexo + DGI + eFacturaUY */ }  
  
// Mezclar la pasarela argentina con el validador uruguayo compila perfecto y factura mal. La fábrica lo hace imposible.
```

DECISIÓN DE CÁTEDRA En el FitZone de este cuatrimestre hay una sola sede país y una sola pasarela: Abstract Factory NO se justifica. Escribir en el ADR «se evaluó Abstract Factory y se descartó porque no existe una segunda familia» vale lo mismo que aplicarlo bien.

Builder: el constructor telescópico y su cura

Intención: separar la construcción de un objeto complejo de su representación, para que el mismo proceso arme distintas configuraciones.

ANTES · constructor telescópico

```
new Reserva(socio, cancha, franja,
            null,          // cupón: no aplica
            true,          // ¿es socio?
            false,         // ¿horario pico?
            true,          // ¿permite cancelar?
            "WEB",         // origen
            null);         // observaciones

// ¿Cuál de los tres boolean era el pico?
// Invertí dos y el sistema cobra mal.
// Compila igual. Los tests pasan si no
// se te ocurrió escribir ESE test.
```

DESPUÉS · Builder

```
Reserva r = Reserva.para(socio)
    .enCancha(cancha)
    .enFranja(franja)
    .conCupon(cupon)           // opcional
    .origen(Origen.WEB)
    .construir();              // acá valida

// Cada llamada se lee sola.
// Lo opcional se omite en vez de ir null.
// esSocio y esHorarioPico NO se pasan:
// se derivan del socio y de la franja.
// El objeto nace válido o no nace.
```

EL BENEFICIO REAL No es la sintaxis encadenada: es que `construir()` es el único lugar donde se validan los invariantes del objeto. «Una reserva siempre tiene socio, cancha y franja; el cupón solo se acepta si el socio está al día» se escribe una vez, ahí, y ningún camino del código puede saltarlo.

Cuándo sí

Cuatro o más parámetros, varios opcionales, o dos del mismo tipo seguidos (dos `LocalDateTime`, dos `String`).

Cuándo no

Tres campos obligatorios y ningún opcional: un record de Java 21 alcanza y sobra. El Builder ahí es ruido.

Trampa frecuente

Un Builder que no valida nada en `construir()`. Es un constructor telescópico con más código y la misma fragilidad.

El mismo patrón, tres idiomas distintos

Java con record y builder anidado · Java con Lombok · TypeScript con objeto de opciones y tipos discriminados.

Java 21 · builder a mano

```
public record Reserva(Socio socio,
    Cancha cancha, Franja franja,
    Cupon cupon, Origen origen) {

    public static Builder para(Socio s) {
        return new Builder(s);
    }

    public static final class Builder {
        private final Socio socio;
        private Cancha cancha;
        private Franja franja;
        private Cupon cupon;
        private Origen origen = Origen.WEB;
        Builder(Socio s){ this.socio = s; }

        public Builder enCancha(Cancha c) {
            this.cancha = c; return this; }
        // ... enFranja, conCupon, origen

        public Reserva construir() {
            requireNonNull(cancha, "cancha");
            if (cupon != null && socio.enMora())
                throw new CuponNoAplicable();
            return new Reserva(socio, cancha,
                franja, cupon, origen);
        } } }
```

Java · Lombok

```
@Builder
public record Reserva(
    @NonNull Socio socio,
    @NonNull Cancha cancha,
    @NonNull Franja franja,
    Cupon cupon,
    @Builder.Default
    Origen origen = Origen.WEB) {

    // el builder se genera solo...
    public Reserva {
        // ...y el constructor compacto
        // sigue siendo el único lugar
        // donde vive el invariante
        if (cupon != null && socio.enMora())
            throw new CuponNoAplicable();
    }

    Reserva.builder()
        .socio(s).cancha(c)
        .franja(f).build();

    // Cuidado: @Builder sin
    // constructor compacto genera
    // objetos inválidos con toda
    // comodidad.
```

TypeScript · opciones + esquema

```
const ReservaSchema = z.object({
    socioId: z.string().uuid(),
    canchaId: z.string().uuid(),
    franja: FranjaSchema,
    cupon: CuponSchema.optional(),
    origen: z.enum(['WEB', 'APP', 'MOSTRADOR'])
        .default('WEB'),
}).refine(
    r => !r.cupon || !enMora(r.socioId),
    { message: 'cupón no aplicable en mora' }
);

type Reserva = z.infer<typeof ReservaSchema>;

export function crearReserva(
    e: unknown): Reserva {
    return ReservaSchema.parse(e); // valida
}

crearReserva({
    socioId, canchaId, franja, // el orden
}); // no importa

// Sin encadenamiento: el objeto
// literal ya es legible y el
// esquema valida.
```

LO QUE HAY QUE LLEVARSE Las tres versiones cumplen la misma intención: que no exista ningún camino para construir una Reserva inválida. La estructura de GoF es una forma de conseguirlo, no la definición del objetivo.

ENLACE NestJS 12 valida @Body con Standard Schema (Zod, Valibot): el mismo esquema que construye el objeto valida la petición HTTP. Un único punto de verdad para el invariante.

Singleton: el patrón más conocido y el peor recomendado

Intención: garantizar que una clase tenga una sola instancia y proveer un punto de acceso global a ella.

La forma más segura en Java

```
public enum ConfiguracionGlobal {  
    INSTANCIA;  
    private final Properties props = cargar();  
    public String get(String clave) {  
        return props.getProperty(clave);  
    }  
}  
  
// El enum resuelve gratis la inicialización  
// perezosa segura entre hilos y la  
// serialización. El doble bloqueo con  
// volatile es la versión de examen; esta  
// es la que se escribe.
```

1

Rompe los tests

El estado sobrevive entre casos de prueba: el test 7 falla por lo que hizo el test 3. Y no se puede sustituir por un doble.

2

Miente sobre las dependencias

Una clase que usa Config.INSTANCIA no lo declara en su constructor. La dependencia existe, pero es invisible: es un Service Locator.

3

Concurrencia

Una sola instancia con estado mutable compartida entre hilos. Es el Caso 2 de la Unidad II (ConsultaYa ·

4

Es estado global con buenos modales

El problema de fondo no es la instancia única: es que cualquiera, desde cualquier

DISTINGUIR DOS COSAS «Que exista una sola instancia» es un requerimiento razonable y frecuente. «Que cualquiera pueda alcanzarla desde cualquier lado con una llamada estática» es el problema. El patrón de GoF acopla las dos cosas, y de ahí viene todo el daño.

LO QUE SÍ ES SINGLETON LEGÍTIMO Objetos inmutables y sin estado: una configuración de solo lectura, una constante compleja, un registro de metadatos calculado al arrancar. Sin estado mutable, tres de los cuatro problemas desaparecen.

PRÓXIMO SLIDE Y sin embargo tu aplicación Spring está llena de singletons. ¿Por qué esos sí?

Todo bean de Spring es un singleton — y ninguno tiene estos problemas

La diferencia no está en cuántas instancias hay: está en quién las administra y cómo llegan al que las usa.

Dimensión	Singleton clásico (GoF)	Bean singleton del contenedor
Alcance de la unicidad	Una instancia por cargador de clases de la JVM.	Una instancia por contenedor. Los tests levantan el suyo.
Cómo llega al cliente	El cliente la busca: Config.INSTANCE. Dependencia oculta.	El contenedor la inyecta por constructor. Dependencia declarada.
Sustitución en pruebas	Imposible sin reflexión o sin un «setter de test» vergonzante.	@MockitoBean, un perfil distinto o un new con un doble.
Ciclo de vida	Estático. Se crea al cargar la clase y muere con la JVM.	Administrado: @PostConstruct, @PreDestroy, refresco del contexto.
Estado mutable	Compartido entre todos los hilos, sin ninguna red de contención.	Igual de peligroso: el scope no sincroniza nada. El bean debe ser sin estado.

EL ÚNICO PROBLEMA QUE EL CONTENEDOR NO RESUELVE El estado mutable en un bean singleton sigue siendo una condición de carrera. Spring no sincroniza nada por vos. Un contador de aforo como campo de un @Service es exactamente el mismo error, con anotación.

El error, con anotación

```
@Service
public class AforoService {
    private int adentro = 0;
    public void ingresa() { adentro++; }
}
```

// un solo bean, N hilos de petición: esto se corrompe
// ni siquiera es atómico: leer, sumar, escribir

Prototype, y los errores que se repiten todos los años

Prototype · intención

Crear objetos nuevos copiando un ejemplar ya configurado, en vez de construirlos desde cero. Útil cuando la configuración inicial es cara o viene de datos.

Prototype · en FitZone

La «semana tipo» de una sede: 40 franjas con instructores y cupos. Dar de alta la semana siguiente es clonar la plantilla y ajustar dos horarios.

Prototype · la trampa

Copia superficial. Al clonar la grilla, las 40 franjas siguen apuntando a los MISMOS objetos: editar una de la semana nueva edita la de la plantilla.

Java · copia profunda explícita

```
public GrillaSemanal copiaPara(LocalDate lunes) {
    var franjas = this.franjas.stream()
        .map(f -> f.copiaEn(lunes))    // cada hija
        .toList();                     // se copia
    return new GrillaSemanal(lunes, franjas);
} // sin Cloneable: un método de dominio y listo
```

TypeScript

```
const nueva = structuredClone(plantilla); // profunda
const superficial = { ...plantilla };      // NO lo es:
// superficial.franjas es el MISMO arreglo.
// Es la causa número uno de «se me editó el
// original» en los trabajos de la cátedra.
```

Error frecuente	Por qué aparece	Qué hacer
Una fábrica para una sola clase concreta	Se aplicó el patrón por costumbre, no por variación.	Borrarla. Volver al new o al contenedor.
Singleton a mano dentro de Spring	Se aprendió el patrón antes que el contenedor.	Un @Component sin estado.
Builder que no valida	Se copió la sintaxis encadenada sin la intención.	Poner los invariantes en construir().
Clonado superficial tomado por profundo	El spread y clone() se parecen a una copia y no lo son.	Copiar las hijas o usar structuredClone.

¿Qué creacional — o ninguno?

De a dos, un minuto por caso. Se responde con el patrón y con una razón; «ninguno» es una respuesta válida y a veces la correcta.

- 1 La pasarela de pago se elige por entorno:** en desarrollo un simulador, en producción MercadoPago. Nunca cambia durante la ejecución.
- 2 El gerente exporta el reporte de ocupación** eligiendo el formato en la interfaz: PDF, XLSX o CSV. Se esperan más formatos el cuatrimestre próximo.
- 3 Alta de una promoción:** nombre, porcentaje, sedes alcanzadas, tipos de cancha, fecha desde, fecha hasta, tope de usos y si acumula con la de socio.
- 4 Al crear una sede nueva** hay que darle la misma grilla horaria que usa la sede Central, que está cargada en la base de datos.

LA PREGUNTA QUE ORDENA TODO ¿La decisión depende de un dato que solo existe en tiempo de ejecución? Si no, no hay fábrica que valga: hay configuración. Dos de los cuatro casos de arriba se resuelven sin ningún patrón.

2

BLOQUE 2 · 55 MINUTOS

Patrones estructurales

Siete maneras de componer objetos. Cuatro de ellas son literalmente «un objeto que envuelve a otro» — y se distinguen únicamente por para qué lo envuelven.

Casi todos envuelven algo. La diferencia es para qué

Misma estructura, cuatro intenciones distintas. Esta tabla es la respuesta a la pregunta de parcial más frecuente de la unidad.

Patrón	¿Cambia la interfaz?	¿Cambia el comportamiento?	Intención en una oración
Adapter	Sí, la traduce	No	Hacer que dos interfaces incompatibles trabajen juntas.
Decorator	No	Sí, agrega	Sumar responsabilidades a un objeto dinámicamente, sin tocar su clase.
Proxy	No	No (controla)	Controlar el acceso al objeto real: retrasarlo, cachearlo, protegerlo, remotizarlo.
Facade	Sí, la simplifica	No	Ofrecer una entrada única y sencilla a un subsistema complicado.
Composite	Unifica hoja y rama	—	Tratar objetos individuales y composiciones de objetos de la misma manera.
Bridge	Separa dos jerarquías	—	Desacoplar una abstracción de su implementación para que varíen por separado.
Flyweight	—	—	Compartir el estado común entre muchísimos objetos para que entren en memoria.

1

El envoltorio obligado

Adapter: la interfaz de afuera no es la que necesitás. Se traduce en la frontera y el dominio ni se entera.

2

El envoltorio que suma

Decorator: la interfaz está bien, falta comportamiento. Se apilan varios y el orden importa.

3

El envoltorio invisible

Proxy: el cliente cree que habla con el objeto real. Es el que usa tu framework sin avisarte.

Adapter: el traductor de la frontera

Intención: convertir la interfaz de una clase en otra que el cliente espera, para que puedan colaborar clases que no fueron pensadas juntas.

El puerto: lo define el dominio

```
// paquete: dominio.pagos (no conoce a nadie)
public interface PasarelaPago {
    ResultadoPago debitar(Monto m, TokenTarjeta t);
}

public record Monto(BigDecimal valor, Moneda moneda) {}

public sealed interface ResultadoPago
    permits Aprobado, Rechazado, Pendiente {}

// El dominio habla de plata, no de centavos;
// de resultados, no de códigos numéricos;
// y nunca, jamás, de MercadoPago.
```

El adaptador: lo sucio, en un solo archivo

```
// paquete: infraestructura.pagos.mercadopago
@Component @Profile("prod")
class MercadoPagoAdapter implements PasarelaPago {
    private final MP_Client sdk; // el de ellos

    public ResultadoPago debitar(Monto m, TokenTarjeta t) {
        var centavos = m.valor()
            .movePointRight(2).longValueExact();
        try {
            var r = sdk.createPayment(centavos,
                m.moneda().name(), t.valor());
            return switch (r.getStatus()) {
                case "approved" -> new Aprobado(r.getId());
                case "in_process" -> new Pendiente(r.getId());
                default -> new Rechazado(r.getStatusDetail());
            };
        } catch (MPApiException e) { // y el error
            throw new PagoNoDisponible(e); // también se traduce
        }
    }
}
```

LO QUE COMPRA EL ADAPTADOR Tres cosas concretas: (1) el dominio se prueba con un doble del puerto, sin red; (2) cambiar de pasarela toca un archivo, no cuarenta; (3) la excepción propietaria del SDK no se filtra a la capa de negocio, que es como un cambio de versión del SDK termina rompiendo un controlador.

ENLACE CON UNIDAD I Puerto + adaptador = arquitectura hexagonal. Lo que en Unidad I era una decisión de diagrama C4, acá es una interfaz y una clase. El patrón de GoF es el mecanismo con el que se implementa el estilo arquitectónico.

Decorator: apilar responsabilidades sin explotar en subclasses

Intención: agregar responsabilidades a un objeto dinámicamente, como alternativa flexible a la herencia.

Los decoradores

```
public interface CalculoPrecio {
    BigDecimal total(Reserva r);
}

class PrecioBase implements CalculoPrecio { ... }

abstract class Modificador implements CalculoPrecio {
    protected final CalculoPrecio siguiente;
    Modificador(CalculoPrecio s){ this.siguiente = s; }
}

class DescuentoSocio extends Modificador {
    public BigDecimal total(Reserva r) {
        var base = siguiente.total(r);           // delega
        return r.socio().alDia()
            ? base.multiply(new BigDecimal("0.85"))
            : base;                             // y decora
    }
}
```

El apilado — y el orden importa

```
CalculoPrecio calculo =
    new Iva(
        new Cupon(
            new RecargoHorarioPico(
                new DescuentoSocio(
                    new PrecioBase(tarifa))))));

// Se lee de adentro hacia afuera:
//   base → descuento socio → recargo pico
//   → cupón → IVA

// Mover Iva al centro cambia la factura.
// Esa decisión es del negocio, y acá queda
// escrita en una sola expresión legible
// en vez de enterrada en un método de 80
// líneas con seis if anidados.
```

Con herencia: $2^4 = 16$ subclasses

- PrecioConSocio, PrecioConSocioYPico, PrecioConSocioYPicoYCupon...
- El quinto modificador duplica la jerarquía: 32 clases.
- La combinación se fija al compilar; el socio no puede elegir.

Con Decorator: 4 clases

- Una clase por modificador, componibles en cualquier orden.
- El quinto modificador es una clase más. Nada existente se edita.
- La combinación se arma en ejecución, según la reserva concreta.

Decorator, tres veces por día, sin darte cuenta

JDK

```
var in = new BufferedReader(  
    new InputStreamReader(  
        new GZIPInputStream(  
            new FileInputStream(f))));  
  
// leer archivo  
//   → descomprimir  
//   → decodificar a caracteres  
//   → almacenar en buffer  
  
// Cada capa implementa la misma  
// interfaz que envuelve. Es el  
// ejemplo canónico del libro.
```

Spring

```
@Component  
class MedidorDeTiempo  
    implements BeanPostProcessor {  
    public Object postProcessAfterInit(  
        Object bean, String nombre) {  
        // devuelve un envoltorio del bean  
        return envolver(bean);  
    }  
}  
  
// La cadena de filtros de Spring  
// Security es la misma idea:  
// cada filtro decide si delega.
```

TypeScript / React

```
const conReintento =  
    (fn: Fetcher): Fetcher =>  
        async (url) => {  
            try { return await fn(url); }  
            catch { return await fn(url); }  
        };  
  
const cliente = conCache(  
    conReintento(  
        conAuth(fetchBase)));  
  
// Mismo patrón, sin clases:  
// funciones que envuelven  
// funciones del mismo tipo.
```

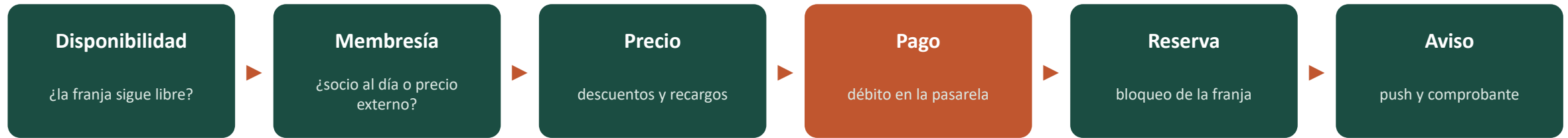
LA MARCA DEL PATRÓN Si el envoltorio y lo envuelto tienen el mismo tipo, y el envoltorio delega y agrega, es un Decorator — tenga clases, funciones o anotaciones. Si el envoltorio expone un método que el envuelto no tiene, es otra cosa (y probablemente sea un Adapter mal hecho).

EL COSTO, QUE ES REAL Depurar una pila de seis decoradores es incómodo: la traza de excepción tiene seis marcos que no dicen nada y ninguna clase «hace» el trabajo completo. Por eso conviene que el armado de la pila viva en un solo lugar visible y documentado, no repartido.

EN FITZONE RF-11 (precio dinámico) admite Decorator o Strategy. Elegir uno y justificar el descarte del otro es exactamente lo que pide el entregable: se vuelve sobre esto en el Bloque 3.

Facade: una puerta simple para un subsistema que no lo es

Intención: proveer una interfaz unificada de alto nivel sobre un conjunto de interfaces de un subsistema.



La fachada orquesta (y compensa)

```

@Service
public class ReservaDeCanchas {           // la fachada
    @Transactional
    public Comprobante reservar(SolicitudReserva sol) {
        var franja = disponibilidad.tomar(sol.franjaId());
        var socio = membresias.estadoDe(sol.socioId());
        var precio = tarifas.calcular(franja, socio);
        var pago = pasarela.debitar(precio, sol.token());
        if (pago instanceof Rechazado r) {
            disponibilidad.liberar(franja); // compensación
            throw new PagoRechazado(r.motivo());
        }
        var reserva = reservas.confirmar(franja, socio, pago);
        eventos.publicar(new ReservaConfirmada(reserva.id()));
        return comprobantes.emitir(reserva);
    }
}
  
```

Lo que gana el cliente

El controlador REST hace una llamada y no conoce ni el orden, ni la compensación, ni los cinco colaboradores. La app móvil tampoco.

Facade no oculta: simplifica

Los subsistemas siguen siendo públicos: quien necesite el detalle puede saltar la fachada. Si la fachada es obligatoria, es otra capa, no un Facade.

Facade vs. Service Layer

Service Layer (Fowler) es una capa arquitectónica con transacciones y seguridad; Facade es un patrón de objeto. En Spring coinciden en la misma clase, está bien, pero son dos ideas.

PARA EL TRABAJO INTEGRADOR El método reservar() de arriba es el corazón del caso FitZone y el lugar donde aparece RN-02 (dos usuarios, la misma cancha, el mismo segundo). Quién gana esa carrera no lo resuelve el patrón: lo resuelve el bloqueo optimista con @Version, que se ve en la Clase 2 junto con Repository.

Proxy: el objeto que se hace pasar por otro

Intención: proveer un sustituto o representante de otro objeto para controlar el acceso a él.

1 Proxy virtual

Difiere la creación o la carga del objeto real hasta que se lo usa.

Hibernate: una colección
@OneToMany perezosa es un proxy.
Tocarla fuera de la sesión lanza
LazyInitializationException.

2 Proxy de protección

Controla quién puede invocar qué.

Spring Security:
@PreAuthorize("hasRole('GERENTE')")
es un proxy que decide si deja pasar la llamada al método.

3 Proxy remoto

Representa localmente a un objeto que vive en otro proceso o máquina.

Un cliente declarativo de HTTP: la interfaz parece local, la llamada viaja por la red.

4 Proxy inteligente

Agrega tareas al acceso: contar, cachear, abrir transacción.

@Transactional y @Cacheable son exactamente esto, y son la razón del problema del próximo slide.

Lo que Spring construye cuando ve @Transactional

<pre>@Service public class ReservaService { @Transactional public void confirmar(Long id) { ... } } // El bean inyectado en el controlador NO es // tu clase: es un subtipo generado en caliente</pre>	// Lo que escribís: // // ReservaService → // //	Lo que hay en el contenedor: <pre>\$Proxy42 implements ... { confirmar(id) { tx.begin(); real.confirmar(id); // delega tx.commit(); } }</pre>
---	---	--

CONSECUENCIA PRÁCTICA Si el bean inyectado no es tu clase sino un subtipo generado, todo lo que dependa de la identidad de la clase se comporta distinto: los métodos private y final no se pueden interceptar, y la llamada interna no pasa por el proxy. Eso último es el error más frecuente de la unidad, y va aparte.

Por qué tu @Transactional «no anda»

La autoinvocación. Aparece todos los años en algún Trabajo Integrador, y es una pregunta habitual en entrevistas técnicas.

NO FUNCIONA

```
@Service
public class ReservaService {

    public void confirmarLote(List<Long> ids) {
        for (Long id : ids) {
            confirmar(id);    // ← llamada interna:
        }                    // this es el objeto real,
                            // no el proxy

    @Transactional
    public void confirmar(Long id) { ... }
}

// No hay error. No hay excepción. No hay
// transacción tampoco. Si el ítem 7 falla,
// los seis anteriores quedan escritos.
```

FUNCIONA · extraer a otro bean

```
@Service
public class ConfirmacionDeReserva {    // bean aparte
    @Transactional
    public void confirmar(Long id) { ... }
}

@Service
public class ReservaService {
    private final ConfirmacionDeReserva confirmacion;

    public void confirmarLote(List<Long> ids) {
        ids.forEach(confirmacion::confirmar);
    }    // ← ahora sí pasa por el proxy del otro bean
}

// Efecto secundario deseable: la separación
// suele revelar que eran dos responsabilidades.
```

Salida	Cómo	Veredicto
Extraer a otro bean	La lógica anotada vive en un colaborador inyectado.	Recomendada. Es la que se espera en el entregable.
Autoinyección	Inyectarse a sí mismo y llamar self.confirmar(id).	Funciona; delata el problema de diseño en vez de resolverlo.
TransactionTemplate	Abrir la transacción a mano, sin anotación.	Válida cuando el alcance transaccional es dinámico.
AspectJ en carga de clase	Tejido real de bytecode: no hay proxy que saltar.	Potente y desproporcionada para una aplicación de cátedra.

LO MISMO PASA CON @Cacheable, @Async, @Retryable, @PreAuthorize y @Validated. No es una rareza de las transacciones: es cómo funciona un proxy.

Composite: que el cliente no tenga que preguntar «¿es una hoja o una rama?»

Intención: componer objetos en estructuras de árbol y tratar de manera uniforme a los objetos individuales y a sus composiciones.

La estructura

```
public interface UnidadOrganizativa {    // el componente
    String nombre();
    int ocupacionActual();
    int aforoMaximo();
    default double porcentajeUso() {
        return aforoMaximo() == 0 ? 0
            : 100.0 * ocupacionActual() / aforoMaximo();
    }
}

record Espacio(String nombre, int ocupacionActual,
    int aforoMaximo) implements UnidadOrganizativa {} // hoja

record Agrupacion(String nombre,                // la rama
    List<UnidadOrganizativa> hijas)
    implements UnidadOrganizativa {
    public int ocupacionActual() { return hijas.stream()
        .mapToInt(UnidadOrganizativa::ocupacionActual).sum(); }
    public int aforoMaximo() { ... }
}
```

El árbol de FitZone

Red FitZone → Región Litoral → Sede Paraná Centro → Cancha de paddle 3.
Cuatro niveles, un solo tipo.

Lo que gana el cliente

unidad.porcentajeUso() responde igual si «unidad» es una cancha o la red completa. Desaparece todo instanceof y toda rama del recorrido.

El cuidado nº1

La recursión toca todos los nodos: si cada uno consulta la base, es un N+1 en forma de árbol. Hay que cargar el subárbol de una vez.

EL OLOR QUE ELIMINA Si el código del gerente dice if (nodo instanceof Sede s) { ... } else if (nodo instanceof Cancha c) { ... }, falta un Composite. Cada tipo nuevo de unidad agrega una rama a ese if, en cada lugar donde se recorre el árbol.

Patrón	Intención	Dónde aparece de verdad
Bridge	Separar una abstracción de su implementación para que las dos jerarquías varíen sin multiplicarse.	Notificación (aviso de clase, aviso de mora) × canal (push, e-mail, SMS): 2+3 clases en vez de 2×3.
Flyweight	Compartir el estado intrínseco entre muchísimos objetos para que entren en memoria.	Raro en aplicaciones de gestión. Habitual en juegos, editores de texto y renderizado. Integer.valueOf() lo usa.

3

BLOQUE 3 • 70 MINUTOS

Patrones de comportamiento

Once patrones sobre cómo se reparte la responsabilidad y cómo se comunican los objetos. Es el grupo más grande, el más usado y el que más aparece en el Trabajo Integrador.

Los once de comportamiento

Seis se ven con código. Los otros cinco quedan ubicados, con el lugar donde de verdad aparecen.

Patrón	Intención en una oración	En FitZone / en el stack
Strategy ★	Encapsular algoritmos intercambiables y elegirlos en ejecución.	RF-11: precio estándar, descuento de socio, recargo por horario pico.
Observer ★	Notificar a varios interesados cuando algo cambia, sin que el emisor los conozca.	RF-08: al liberarse un cupo, avisar a la lista de espera.
Command ★	Encapsular una petición como objeto: encolarla, registrarla, deshacerla.	Operaciones de mostrador con deshacer; tareas diferidas; CQRS en la Clase 2.
Template Method ★	Fijar el esqueleto de un algoritmo y delegar pasos en las subclases.	Emisión de comprobantes: el flujo es uno, el formato cambia. JdbcTemplate.
State ★	Cambiar el comportamiento de un objeto cuando cambia su estado interno.	Membresía: Activa / Vencida / Suspendida. Cancha: Disponible / En mantenimiento.
Chain of Responsibility ★	Pasar una petición por una cadena hasta que alguien la atiende.	Validaciones previas a la reserva. Filtros HTTP de Spring Security.
Iterator	Recorrer una colección sin exponer su representación interna.	Absorbido por el lenguaje: for-each, Iterable, Stream, generadores de JS.
Mediator	Centralizar la comunicación entre objetos que si no se hablarían todos con todos.	Un bus de eventos en memoria; el patrón detrás de muchos «event bus» del frontend.
Memento	Guardar y restaurar el estado interno de un objeto sin violar su encapsulamiento.	Deshacer; instantáneas de un formulario largo; borradores.
Visitor	Agregar operaciones nuevas a una estructura de objetos sin modificar sus clases.	Recorrido de árboles: informes sobre el Composite de sedes; análisis de código.

PAR QUE SE CONFUNDE SIEMPRE Strategy y State tienen el MISMO diagrama de clases. En Strategy, el cliente elige la estrategia y esta no sabe que hay otras. En State, el propio estado decide cuál es el siguiente. Volvemos sobre esto con código.

Strategy: el if/else que crece con cada regla de negocio

Intención: definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. RF-11 del caso FitZone.

ANTES · el método que factura, editado todos los meses

```
public BigDecimal precio(Reserva r) {
    BigDecimal p = r.cancha().tarifaBase();

    if (r.socio() != null && r.socio().alDia()) {
        p = p.multiply(new BigDecimal("0.85")); // RF-11
    }
    if (r.franja().hora() >= 19 && r.franja().hora() < 21) {
        p = p.multiply(new BigDecimal("1.20"));
    }
    if (r.socio() != null && !r.socio().alDia()) {
        p = r.cancha().tarifaExterno(); // RN-03
    }
    if (r.cancha().tipo() == TipoCancha.FUTBOL_5
        && r.franja().esFinDeSemana()) {
        p = p.add(new BigDecimal("2500"));
    }
    // ... y el martes que viene, la promo 2x1
    return p;
}
```

1

Viola OCP

Cada regla nueva modifica un método que ya funcionaba y que, además, es el que calcula el precio.

2

El orden es lógica invisible

¿El recargo se aplica sobre el precio ya descontado? Lo decide el orden de los if, y no está escrito en ningún lado.

3

No se puede probar una regla sola

Para verificar el recargo de horario pico hay que construir una Reserva completa y pasar por las otras seis ramas.

4

No se puede configurar

EL OLOR TIENE NOMBRE «Sentencia switch» / «condicional complejo» en el catálogo de Fowler. El refactoring que corresponde se llama Replace Conditional with Polymorphism y su destino natural es Strategy. Es el camino que se recorre en el laboratorio de la Clase 2.

PERO ANTES Si hubiera solo dos reglas y no se esperaran más, el if está bien. La regla de tres también vale acá.

Strategy: una clase por regla, y el contenedor las junta

DESPUÉS · Java 21 / Spring Boot 4

```
public interface ReglaDePrecio {
    boolean aplica(Reserva r); // ¿me toca?
    BigDecimal aplicar(BigDecimal actual, Reserva r);
}

@Component @Order(10)
class DescuentoSocio implements ReglaDePrecio {
    public boolean aplica(Reserva r) {
        return r.socio() != null && r.socio().alDia();
    }
    public BigDecimal aplicar(BigDecimal p, Reserva r) {
        return p.multiply(new BigDecimal("0.85"));
    }
}

@Component @Order(20)
class RecargoHorarioPico implements ReglaDePrecio { ... }

@Service
public class Tarifador {
    private final List<ReglaDePrecio> reglas; // Spring las
    // inyecta todas, ya ordenadas
    public BigDecimal precio(Reserva r) {
        return reglas.stream().filter(x -> x.aplica(r))
            .reduce(r.cancha().tarifaBase(),
                (p, regla) -> regla.aplicar(p, r),
                (a, b) -> b);
    }
}
```

La promo 2x1 del martes

Una clase nueva con `@Component` y `@Order(30)`. Ningún archivo existente se edita, ningún test existente se rompe. Eso es OCP en la práctica.

Cada regla se prueba sola

`new DescuentoSocio().aplicar(mil, reserva)` es un test unitario de tres líneas, sin contenedor, sin base de datos y sin red.

El orden dejó de ser accidental

`@Order(10)` antes que `@Order(20)` es una decisión de negocio escrita y revisable. Antes era la posición de un `if`.

El costo

Cuatro archivos donde había un método. Con dos reglas no vale la pena; con siete, y una nueva por mes, sí. Ese cálculo hay que escribirlo en el ADR.

ENLACE CON UNIDAD II Es el Caso 3 de la ejercitación de IoC (RutaYa · AsignadorDeEnvios). En Spring el tipo es el token y `List<ReglaDePrecio>` se resuelve sola; en NestJS el token de colección hay que declararlo con `useFactory`. El patrón es el mismo, el contenedor no.

VARIANTE ÚTIL Si las reglas vienen de la base de datos (porcentaje por sede), la estrategia se construye con sus parámetros y la fábrica del Bloque 1 vuelve a aparecer: Strategy + Factory Method es una de las combinaciones más frecuentes del catálogo.

Cuatro formas de escribir un Strategy (y una de no escribirlo)

1 · Lambda

```
Map<TipoPlan, UnaryOperator<BigDecimal>>
reglas = Map.of(
    MENSUAL,    p -> p,
    TRIMESTRAL, p -> p.multiply(
        new BigDecimal("0.95")),
    ANUAL,      p -> p.multiply(
        new BigDecimal("0.85")));

var precio = reglas.get(plan).apply(base);

// Estrategia sin estado y sin dependencias:
// una interfaz funcional alcanza. Una clase
// acá sería ceremonia pura.

// Límite: no se puede inyectar, no se puede
// nombrar en una traza de error y no admite
// configuración por sede.
```

2 · Enum con comportamiento

```
public enum TipoPlan {
    MENSUAL { BigDecimal f(BigDecimal p)
        { return p; } },
    TRIMESTRAL { BigDecimal f(BigDecimal p)
        { return p.multiply(
            new BigDecimal("0.95")); } },
    ANUAL { BigDecimal f(BigDecimal p)
        { return p.multiply(
            new BigDecimal("0.85")); } };
    abstract BigDecimal f(BigDecimal p);
}

// Conjunto CERRADO y conocido: el compilador
// obliga a implementar todas las constantes.
// Ideal para planes, estados y monedas.
```

3 · Clase inyectable · 4 · React

```
// 3 · cuando la estrategia necesita
//    colaboradores o configuración
@Component @Order(10)
class DescuentoSocio implements ReglaDePrecio {
    private final ParametrosSede params; // ←
    ...
}

// 4 · en el frontend, es una prop
type Orden = (a: Turno, b: Turno) => number;

const porHorario: Orden =
    (a, b) => a.inicio - b.inicio;

<Grilla turnos={turnos} orden={porHorario} />

// Comparator de Java y la prop de React
// son el mismo patrón en dos lenguajes.
```

5 · LA QUINTA FORMA: NO ESCRIBIRLO Dos alternativas, estables, sin dependencias y sin configuración: un switch expression de Java 21 sobre un sealed interface es exhaustivo, lo verifica el compilador y se lee de un vistazo. Strategy se justifica cuando las alternativas crecen, necesitan colaboradores, se configuran o las define alguien que no es quien escribe el despachador.

RF-11: ¿STRATEGY O DECORATOR? La pregunta del entregable. Si las reglas se aplican de a una y son excluyentes (esta reserva es de socio O de externo), es Strategy. Si se ACUMULAN sobre el mismo precio (descuento y además recargo y además IVA), la intención es Decorator. En FitZone se acumulan: la implementación con List<ReglaDePrecio> y reduce es, en rigor, una cadena de decoradores armada por el contenedor. Cualquiera de los dos nombres se acepta si la justificación dice esto.

Observer: el que avisa no necesita saber a quién

Intención: definir una dependencia uno-a-muchos, de modo que cuando un objeto cambia de estado todos sus dependientes se enteran automáticamente. RF-08 del caso FitZone.

Sin Observer: cancelar sabe demasiado

- cancelar() llama a listaDeEspera, push, mail, auditoría y métricas.
- Cada interesado nuevo edita cancelar() — el quinto requerimiento, el quinto cambio.
- Si falla el envío de un mail, se cae la cancelación.
- El test de cancelar() necesita cinco dobles.

Con Observer: cancelar publica un hecho

- cancelar() publica ReservaCancelada y termina.
- Cada interesado es una clase aparte que se suscribe. El quinto no toca nada.
- Un oyente que falla no arrastra a los demás (si se maneja bien).
- El test de cancelar() verifica que se publicó el evento. Nada más.

Spring · eventos de dominio

```
@Service class ReservaClaseService {
    private final ApplicationEventPublisher eventos;

    @Transactional
    public void cancelar(Long reservaId) {
        var r = repo.buscar(reservaId).orElseThrow();
        r.cancelar();
        eventos.publishEvent(new CupoLiberado(           // el hecho
            r.claseId(), Instant.now()));                // en pasado
    } // ← y acá termina. No sabe quién escucha.
}
```

Los oyentes

```
@Component class AvisarListaDeEspera {
    @TransactionalEventListener // ← ver próximo
    @Async                      // slide
    public void on(CupoLiberado e) {
        espera.primerode(e.claseId())
            .ifPresent(this::ofrecerCupo);
    }
}

@Component class RegistrarMetrica {
    @EventListener
    public void on(CupoLiberado e) { ... }
}
```

LA INVERSIÓN Antes, el emisor dependía de cinco receptores. Ahora los cinco receptores dependen de un evento. Cambió la dirección de la flecha, y con ella quién tiene que modificarse cuando llega el sexto.

Tres decisiones que Observer obliga a tomar

El patrón es simple; hacerlo bien en un sistema transaccional, no tanto. Estas tres preguntas aparecen sí o sí en el Trabajo Integrador.

1 ¿Antes o después del commit? @EventListener corre dentro de la transacción: si el oyente falla, se revierte la cancelación. @TransactionalEventListener(AFTER_COMMIT) corre después: el hecho ya es firme. Para efectos externos (mail, push, pagos), AFTER_COMMIT.

2 ¿En el mismo hilo o en otro? Por defecto, el mismo: un push lento agrega latencia a la respuesta HTTP del socio que canceló. Con @Async se separa, pero se pierden el contexto de seguridad y la transacción, y las excepciones dejan de propagarse: hay que registrarlas a mano.

3 ¿Y si el proceso se cae entre el commit y el aviso? El evento en memoria se pierde y nadie se entera. Si el aviso es crítico, el patrón no alcanza: hace falta escribir el aviso pendiente en la misma transacción (bandeja de salida) y un proceso que lo despache. Fuera del alcance del TI, pero hay que saber que existe.

4 El riesgo de todo Observer: el flujo deja de leerse. Con diez oyentes, nadie puede decir qué pasa al cancelar una reserva sin buscar por todo el proyecto. Contramedida de cátedra: documentar los eventos y sus oyentes en una tabla del README, y nombrarlos siempre en pasado.

EN EL FRONTEND Mismo patrón, otro riesgo: suscribirse en un useEffect sin devolver la función de limpieza deja el oyente vivo después de desmontar el componente. Es la fuga de memoria más común de React y un clásico de la Unidad IV.

Command: convertir «hacé esto» en un objeto

Intención: encapsular una petición como un objeto, para parametrizar clientes, encolar peticiones, registrarlas y deshacerlas.

La acción como objeto

```
public interface Operacion {
    void ejecutar();
    void deshacer();
    String descripcion(); // para la bitácora
}

record SuspendeMembresia(Long socioId, Motivo motivo,
                        MembresiaRepo repo)
    implements Operacion {
    public void ejecutar() { repo.suspender(socioId, motivo); }
    public void deshacer() { repo.reactivar(socioId); }
    public String descripcion() {
        return "Suspensión de socio " + socioId; }
}

@Service public class Mostrador {
    private final Deque<Operacion> historial = new ArrayDeque<>();
    public void hacer(Operacion op) {
        op.ejecutar(); historial.push(op); }
    public void deshacerUltima() {
        if (!historial.isEmpty()) historial.pop().deshacer(); }
}
```

1

Deshacer

El historial es una pila de comandos. Cada uno sabe revertirse; el mostrador no

2

Encolar y diferir

El comando se serializa y se procesa después: es la forma natural de una tarea

3

Auditar

Registrar quién pidió qué y cuándo sale gratis: el objeto ya contiene la intención

4

Puente a CQRS

El patrón Command es una forma de implementar el patrón CQRS (Command Query Responsibility Segregation)

EL LÍMITE HONESTO DE «DESHACER» deshacer() funciona con cambios internos y reversibles. No se puede deshacer un mail enviado, ni un débito ya acreditado, ni un turno que se abrió. Para esos casos lo que existe es una operación de COMPENSACIÓN (una nota de crédito, no un «borrar el pago»), que es una operación de negocio nueva, no una marcha atrás.

CUÁNDO NO Si no hay deshacer, ni cola, ni auditoría, ni reintento, un Command es una llamada a método con tres archivos de ceremonia alrededor. El patrón se paga con al menos una de esas cuatro necesidades.

Template Method: el esqueleto fijo con los huecos delegados

Intención: definir el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclases, que los redefinen sin cambiar la estructura general.

La plantilla

```
public abstract class EmisorDeComprobante {

    public final Comprobante emitir(Reserva r) { // final:
        validar(r); // el orden no se negocia
        var numero = numerador.siguiente(puntoDeVenta());
        var cuerpo = renderizar(r, numero); // ← hueco
        var guardado = archivo.guardar(cuerpo, extension());
        registrar(r, numero, guardado);
        return new Comprobante(numero, guardado);
    }

    protected abstract byte[] renderizar(Reserva r, String n);
    protected abstract String extension();

    protected void validar(Reserva r) { // gancho con
        if (!r.estaPaga()) throw new NoFacturable(r);
    } // comportamiento
    } // por omisión

    class ComprobantePdf extends EmisorDeComprobante { ... }
    class ComprobanteMail extends EmisorDeComprobante { ... }
```

Template Method • herencia

La variación se elige al COMPILAR: es la subclase. El orden lo fija la clase base y el subtipo no puede alterarlo. Un solo eje de variación.

Strategy • composición

La variación se elige en EJECUCIÓN: es un objeto que se pasa. El orden lo controla el contexto. Varios ejes combinables sin multiplicar clases.

El acoplamiento

El subtipo ve los miembros protegidos del padre y depende de su estructura interna; la estrategia solo conoce una interfaz pública.

CUÁNDO CADA UNO Si lo que hay que garantizar es el ORDEN de una secuencia fija y solo cambia el relleno, Template Method. Si lo que hay que intercambiar es un algoritmo completo y quizás en caliente, Strategy. En el stack: JdbcTemplate y RestTemplate son Template Method; Comparator y PasswordEncoder son Strategy.

VARIANTE MODERNA El mismo efecto sin herencia: el método plantilla recibe los pasos variables como funciones (emitir(Reserva r, Renderizador render)). Se gana composición y se pierde la obligación de implementar todos los huecos, que a veces era justamente lo que se quería. Es una decisión, no una mejora automática.

State: cuando el if pregunta siempre por el mismo campo

Intención: permitir que un objeto altere su comportamiento cuando cambia su estado interno; parecerá que cambió de clase.

ANTES · el estado, disperso en cinco métodos

```
public boolean puedeReservarClase() {
    return estado == ACTIVA; }

public BigDecimal precioDeCancha(Cancha c) {
    if (estado == ACTIVA)      return c.tarifaSocio();
    if (estado == VENCIDA)     return c.tarifaExterno();
    if (estado == SUSPENDIDA) throw new AccesoDenegado();
    return c.tarifaBase(); }

public boolean puedeIngresar() {
    return estado == ACTIVA || estado == VENCIDA; }

public void renovar() {
    if (estado == SUSPENDIDA) throw new RequiereAlta();
    this.estado = ACTIVA; ... }
```

DESPUÉS · cada estado, una clase

```
sealed interface EstadoMembresia
    permits Activa, Vencida, Suspendida {
    boolean puedeReservarClase();
    boolean puedeIngresar();
    BigDecimal precioDeCancha(Cancha c);
    EstadoMembresia alPagar(Pago p); // ← transición
}

record Vencida(LocalDate desde)
    implements EstadoMembresia {
    public boolean puedeReservarClase() { return false; }
    public boolean puedeIngresar()      { return true; }
    public BigDecimal precioDeCancha(Cancha c) {
        return c.tarifaExterno(); } // RN-03
    public EstadoMembresia alPagar(Pago p) {
        return new Activa(p.fecha().plusMonths(1)); }
}
```

LA DIFERENCIA QUE IMPORTA El estado ya no es un dato que todos consultan: es un objeto que decide. Y sealed interface agrega algo que el enum disperso no daba: agregar «DeBaja» rompe la compilación en cada switch que no lo contemple. El olvido silencioso se vuelve imposible.

Estado → evento	Pago acreditado	Vencimiento	Sanción	Alta manual
Activa	Activa (extiende)	Vencida	Suspendida	—
Vencida	Activa	—	Suspendida	—
Suspendida	Suspendida (no aplica)	—	—	Activa

El mismo diagrama, dos patrones distintos

La pregunta de parcial más repetida de la unidad. La respuesta no está en la estructura: está en quién decide y si el objeto sabe que hay otros.

Criterio	Strategy	State
¿Quién elige la implementación?	El cliente o la configuración, desde afuera.	El propio estado: devuelve cuál es el siguiente.
¿La implementación conoce a sus pares?	No. DescuentoSocio no sabe que existe RecargoPico.	Sí, y tiene que conocerlos: Vencida devuelve Activa.
¿Cuántas veces cambia?	Normalmente una, al construir el objeto.	Muchas, a lo largo de la vida del objeto.

¿Qué significa el cambio?	Una decisión técnica o de configuración.	Un hecho del negocio: probablemente haya que auditarlo.
Pregunta rápida	«¿Cómo hago esto?» — hay varias maneras	«¿Qué soy ahora?» — y de eso depende qué puedo hacer

En FitZone es Strategy

- El cálculo del precio: varias reglas para el mismo objetivo.
- La selección del canal de notificación.
- El criterio de ordenamiento de la grilla de turnos.

En FitZone es State

- La membresía: Activa / Vencida / Suspendida (RF-02).
- La cancha: Disponible / En mantenimiento (RF-12).
- La reserva: Pendiente / Confirmada / Cancelada / Cumplida.

Y CONVIVEN El estado de la membresía determina qué regla de precio aplica. State decide qué se puede hacer; Strategy, cómo se hace. En la misma reserva, al mismo tiempo.

Chain of Responsibility, y los cinco que quedan ubicados

Validaciones encadenadas

```
public interface ValidacionDeReserva {
    Optional<Violacion> validar(SolicitudReserva s); }

@Component @Order(10) class FranjaDisponible ...
@Component @Order(20) class SocioSinDeuda ...
@Component @Order(30) class TopeSemanal ...
@Component @Order(40) class CanchaOperativa ... // RF-12

@Service class ValidadorDeReservas {
    private final List<ValidacionDeReserva> cadena;
    public void verificar(SolicitudReserva s) {
        var fallas = cadena.stream()
            .flatMap(v -> v.validar(s).stream()).toList();
        if (!fallas.isEmpty())
            throw new ReservaInvalida(fallas); // todas juntas
    } }
```

La variante importa

GoF: el primero que puede atender, corta. Validación: corren todos y se acumulan las fallas. La segunda sirve para devolver un Problem Details (RFC 9457) con la lista completa, en vez de que el usuario descubra los errores de a

Ya la usás

Los filtros de Spring Security, los middlewares de Express y los interceptores de Axios son Chain of Responsibility.

El riesgo

Que nadie atienda y el fallo sea silencioso. La cadena necesita un eslabón final que siempre responda.

Patrón	Qué resuelve	Dónde aparece de verdad
Iterator	Recorrer una colección sin exponer cómo está guardada por dentro.	Absorbido por el lenguaje: for-each, Iterable, Stream, los generadores de JavaScript. Ya no se escribe a mano.
Mediator	Evitar que N objetos se conozcan todos con todos: hablan a través de un intermediario.	Un bus de eventos en memoria. Cuidado: el mediador tiende a convertirse en un objeto-dios con toda la lógica adentro.
Memento	Guardar y restaurar el estado interno de un objeto sin romper su encapsulamiento.	Deshacer (junto con Command), borradores de formularios largos, instantáneas de configuración.
Visitor	Agregar operaciones nuevas a una estructura de objetos estable sin tocar sus clases.	Recorridos del Composite de sedes para distintos informes. Compiladores y analizadores de código. Caro: agregar un tipo obliga a tocar todos los visitantes.
Interpreter	Definir una gramática pequeña y evaluar sus expresiones.	Motores de reglas configurables por el usuario. Casi siempre conviene una biblioteca existente antes que escribirlo.

4

BLOQUE 4 · 35 MINUTOS

Combinar, criticar y decidir

Los patrones no vienen de a uno, el catálogo tiene treinta años, y la parte difícil no es aplicarlos: es decidir cuándo no.

Los patrones casi nunca vienen solos

En un diseño real, la mitad del catálogo aparece combinada. Estas son las parejas que van a escribir sin pensarlo.

Combinación	Por qué se juntan	En FitZone
Strategy + Factory Method	Alguien tiene que elegir qué estrategia se usa, y esa elección depende de datos de ejecución.	La regla de precio que aplica se determina según el estado de la membresía y la franja.
Strategy + Decorator	Las variantes dejan de ser excluyentes y empiezan a acumularse sobre el mismo valor.	Descuento de socio + recargo de horario pico + cupón + IVA sobre la misma reserva.
Composite + Visitor	Hay un árbol estable y muchas operaciones distintas que recorrerlo.	Sobre el árbol red→región→sede→espacio: informe de ocupación, de facturación, de mantenimiento.
Command + Memento	Para deshacer hace falta la acción y también el estado previo.	Deshacer la última operación de mostrador cuando la acción no es reversible por sí sola.
Observer + Command	El oyente del evento encola un comando en vez de ejecutar el efecto en el acto.	CupoLiberado deja encolado OfrecerCupoAlPrimero, que se reintenta si falla.
Abstract Factory + Singleton	La familia de productos es fija y cara de crear: una sola instancia de la fábrica alcanza.	Con contenedor IoC, esta combinación ya no se escribe: la fábrica es un bean y listo.
Facade + Repository	La fachada orquesta el caso de uso; el repositorio le da la persistencia sin acoplarla.	ReservaDeCanchas sobre ReservaRepository. Se ve completo en la Clase 2.

EL VALOR DEL VOCABULARIO «Una cadena de decoradores de precio, armada por el contenedor y filtrada según el estado de la membresía» describe tres patrones, un requerimiento y una decisión de diseño en veinte palabras. Sin el vocabulario, esa misma frase es un diagrama en el pizarrón y diez minutos de reunión.

PERO Combinar tres patrones para resolver un requerimiento que se resolvía con un método de doce líneas sigue siendo un error, aunque suene impecable en la defensa.

Del olor al patrón: el mapa de refactoring

Los patrones no se aplican al empezar: casi siempre se llega a ellos refactorizando. Esta tabla es el índice de la Clase 2.

Olor de código	Cómo se detecta	Refactoring	Patrón destino
Sentencia switch / condicional repetido	El mismo if sobre el mismo campo, en varios métodos.	Replace Conditional with Polymorphism	Strategy o State
Constructor con demasiados parámetros	Seis o más argumentos; varios null en el punto de llamada.	Introduce Builder / Parameter Object	Builder
Envidia de funcionalidad	Un método usa más datos de otra clase que de la propia.	Move Method	— (lo arregla el movimiento)
Cirugía a escopeta	Un cambio de negocio obliga a editar ocho archivos distintos.	Move Method + Extract Class	Facade, Observer
Jerarquías paralelas	Cada subclase de A obliga a crear una de B.	Replace Inheritance with Delegation	Bridge, Strategy
Código duplicado con variaciones	El mismo algoritmo tres veces, con dos líneas distintas.	Form Template Method	Template Method
Dependencia de una biblioteca externa en el dominio	import com.mercadopago dentro de un @Service.	Extract Interface + Adapter	Adapter
Clase grande / objeto-dios	Una clase con 800 líneas que todos importan.	Extract Class, Extract Delegate	Facade, Mediator
Comentario explicativo largo	Tres líneas de comentario para justificar un bloque.	Extract Method	— (basta con el nombre)

LA TESIS DE KERIEVSKY No se diseña con patrones: se refactoriza HACIA patrones. El punto de partida es el código que ya existe y el olor que se detecta, no un diagrama en una hoja en blanco. La Clase 2 recorre este camino con el código de FitZone en la pantalla.

Cuándo NO usar un patrón

La parte del tema que no está en el índice del libro y es la que separa un buen diseño de una colección de clases.

1

Todavía no hay variación

Una sola implementación y ninguna en el horizonte.

Contrapregunta: ¿podés nombrar el segundo caso concreto? Si no, esperá al tercero.

2

El framework ya lo resuelve

Un Singleton propio con contenedor IoC; un Observer a mano habiendo eventos de dominio.

Contrapregunta: ¿qué me da mi versión que no me da la del framework?

3

El costo supera al beneficio

Cuatro archivos y una interfaz para una regla de dos líneas que no va a cambiar.

Contrapregunta: ¿cuánto código elimina el patrón, y cuánto agrega?

4

La abstracción es prematura

«Duplication is far cheaper than the wrong abstraction» (Sandi Metz).

Contrapregunta: ¿estoy generalizando dos casos que se parecen por casualidad?

5

El equipo no lo va a mantener

Un Visitor impecable en un equipo que nunca lo vio.

Contrapregunta: ¿el próximo que toque esto va a entenderlo sin mí? Es un criterio técnico, no social.

6

El problema es de otro nivel

Ningún patrón GoF arregla un modelo de datos mal normalizado ni un límite de módulo mal trazado.

Contrapregunta: ¿el problema está en el código o en el diseño de más arriba?

EL COSTO REAL DE UN PATRÓN, SIN ADORNOS Archivos nuevos que hay que abrir para entender un flujo · trazas de excepción con marcos que no dicen nada · un salto más en el depurador · una indirección que el que llega nuevo tiene que reconstruir mentalmente · y, la peor, la sensación de que el diseño está terminado porque tiene nombre propio.

LA PREGUNTA QUE RESUME EL BLOQUE ¿Qué cambio futuro, concreto y esperable, vuelve más barato este patrón? Si la respuesta empieza con «por si algún día», el patrón todavía no corresponde.

¿Sigue vigente un catálogo de 1994?

Sí, pero no entero, y no por las razones que suelen darse. Vale la pena discutirlo en vez de repetirlo.

Lo que el tiempo desgastó

- Norvig (1996): 16 de los 23 patrones se vuelven invisibles o triviales en lenguajes con funciones de primera clase. Varios eran parches a C++.
- Iterator lo absorbió el lenguaje; Command y Strategy sin estado son una lambda; Prototype, una función de copia.
- Los creacionales quedaron en gran medida desplazados por el contenedor IoC, que es donde hoy vive la decisión de qué se instancia.
- Singleton está desaconsejado por consenso, y lo dijeron los propios autores.

Lo que sigue en pie

- El vocabulario. Que Strategy sea una lambda no cambia que «esto es un Strategy» siga siendo la forma más rápida de explicarlo.
- Los estructurales y los de comportamiento envejecieron bien: Adapter, Decorator, Proxy, Observer y State describen problemas que el lenguaje no resolvió.
- Las CONSECUENCIAS de cada ficha —qué se gana y qué se paga— siguen siendo el mejor análisis de diseño escrito sobre el tema.
- Los frameworks que usan todos los días están contruidos con el catálogo: leerlos exige conocerlo.

LA CRÍTICA MÁS SERIA, QUE NO ES LA DE NORVIG Que el catálogo se enseña como un objetivo y no como un vocabulario. Aprender los 23 nombres sin aprender a decidir produce exactamente el diseño que el libro quería evitar: sistemas rígidos, esta vez con nombres prestigiosos. GoF dedicó un capítulo entero a «cómo elegir un patrón» y es el que nadie lee.

CONSIGNA OPTATIVA · SUMA EN EL PROCESO Leer el artículo «Design Patterns in Dynamic Languages» de Peter Norvig (1996) y el capítulo 1 de Design Patterns, y escribir una página: ¿cuáles de los 23 seguirían escribiendo hoy en Java 21, y cuáles resolverían con una característica del lenguaje? Se pide una posición y un argumento, no un resumen.

En equipos: elegir, y sobre todo descartar

Equipos de cuatro (los del Trabajo Integrador). Diez minutos de trabajo, cinco de puesta en común. Un requerimiento por equipo.

A Validar una reserva. Ocho reglas hoy, y la sede Norte quiere dos propias. El socio debe recibir TODOS los errores juntos, no de a uno.

B La membresía cambia de situación y con ella cambia qué puede hacer el socio. Auditoría pide saber quién pasó a quién y cuándo.

C Cuando se libera un cupo hay que avisar a la lista de espera, actualizar el tablero de ocupación y registrar la métrica. Mañana se suma facturación.

D El comprobante hoy se emite en PDF. Contaduría pide además un archivo para el sistema fiscal, con los mismos datos, otro formato y el mismo flujo de validación y numeración.

FORMATO DE LA RESPUESTA · ES EL DEL ENTREGABLE (1) Patrón elegido y el requerimiento que lo motiva. (2) Un patrón descartado y por qué. (3) Qué habría que cambiar si el requerimiento creciera. Tres líneas. «Ninguno» sigue siendo una respuesta válida si está defendida.

Para seguir: bibliografía y práctica

Obligatoria

Freeman & Robson, Head First Design Patterns, 2ª ed., 2021 — capítulos 1 a 3 (Strategy, Observer, Decorator).

Gamma, Helm, Johnson & Vlissides, Design Patterns, 1994 — capítulo 1 y las fichas de los trece patrones vistos hoy.

Complementaria

Fowler, Refactoring, 2ª ed., 2018 — catálogo de olores; base de la Clase 2.

Kerievsky, Refactoring to Patterns, 2004.

Fowler, Patterns of Enterprise Application Architecture, 2002 — Repository y Service Layer, para la Clase 2.

Para discutir

Norvig, «Design Patterns in Dynamic Languages», 1996.

Ousterhout, A Philosophy of Software Design, 2018 — la tesis de los «módulos profundos», que discute de frente la fragmentación excesiva.

FIN DE LA CLASE 1

Lo que hay que llevarse de estas cuatro horas

- Un patrón es tres cosas: un nombre compartido, una estructura y —sobre todo— una lista de consecuencias.
- Los 23 son aplicaciones de tres principios: programar contra interfaces, componer antes que heredar, encapsular lo que varía.
- Adapter, Decorator, Proxy y Facade tienen casi el mismo diagrama; la intención es lo único que los distingue. Lo mismo vale para Strategy y State.
- El stack que usan todos los días está construido con el catálogo: @Transactional es un Proxy, BufferedReader un Decorator, List<T> inyectada un Strategy.
- La habilidad que se evalúa no es aplicar un patrón: es justificar por qué ese y no los otros dos — y saber cuándo ninguno.

Clase 2 (2 h): MVC, MVVM, Repository y CQRS · code smells y refactoring hacia patrones · laboratorio sobre el módulo de precios de FitZone.